2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS) | 979-8-3315-8894-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/CSITSS67709.2025.11295611

Efficient 64-Bit Comparator Architectures: Leveraging Adders for Enhanced Speed and Accuracy

T O Geetha Rani
Dept. of ECE
Visvesvaraya Technological University
Siddaganga Institute Of Technology
Tumakuru, India
togeetha@sit.ac.in

Kumar Harsh
Dept. of ECE
Visvesvaraya Technological University
Siddaganga Institute Of Technology
Tumakuru, India
1si21ec050@sit.ac.in

Anuj Kashyap
Dept. of ECE
Visvesvaraya Technological University
Siddaganga Institute Of Technology
Tumakuru, India
1si22ec007@sit.ac.in

***Abstract*—This paper presents the design, synthesis, and analysis of a 64-bit comparator using Cadence tools, focusing on addressing timing challenges and optimizing performance for high-speed applications. The comparator was implemented using a Carry Look-Ahead Adder (CLA) architecture to minimize propagation delays and enable faster computation. A comparison of the 64-bit comparator using Ripple Carry Adder (RCA), Carry Increment Adder (CIA), and CLA revealed that CLA provides the best balance of delay, power, and area, making it the optimal choice. Techniques such as buffer insertion, gate resizing, and clock tree synthesis were employed to enhance timing, area, and power metrics. Static timing analysis and layout optimization were performed using Cadence Innovus and Tempus tools to ensure a reliable and high-performance design. Floor planning and routing strategies were optimized to reduce congestion and improve signal integrity. Simulation results of BRAM data comparison using 64-bit comparator for efficient, high-speed data comparisons validate the comparator's functionality for $A > B$, $A = B$, and $A > B$, demonstrating its reliability and suitability for processors, embedded systems, and real-time computing environments requiring low latency and high accuracy.**

***Index Terms*—64-bit comparator, Cadence tools, static timing analysis, floor planning, RTL synthesis.**

I. Introduction

The 64-bit comparator is a pivotal component in modern digital systems, particularly in processors, embedded systems, and signal processing applications. Its primary function is to compare two 64-bit binary numbers and determine their relative magnitudes—whether one is greater than, equal to, or less than the other. Such comparisons are integral to decision making operations in arithmetic units, control systems, and real-time computing environments. With the increasing complexity and speed requirements of modern processors, ensuring the comparator's precise timing performance has become critical.

In high-performance computing systems, timing violations such as setup and hold time failures can significantly impact the reliability and speed of operations. These issues are further exacerbated by the demand for compact designs with low power consumption. Therefore, optimizing timing, area, and power consumption while maintaining functionality is a significant challenge in comparator design.

This paper presents the design, synthesis, and analysis of a 64-bit comparator using Cadence tools, including Genus for logic synthesis, Innovus for physical implementation, and Tempus for static timing analysis. The comparator employs a Carry Look-Ahead Adder (CLA) architecture to minimize propagation delays and ensure high-speed computation.

II. Literature Survay

Advancements in the design and simulation of comparators have emphasized the importance of optimizing performance metrics using industry-standard tools like Cadence Virtuoso, as explored in [1]. High-speed and area-efficient architectures, such as the 64-bit digital comparator, have demonstrated significant improvements in operational efficiency [2]. The implementation of reversible logic in ASIC-based comparators has further highlighted opportunities for energy-efficient and scalable designs [3]. FPGA-based designs, including 64- bit signed magnitude comparators for IoT applications, have shown promise in addressing power and area constraints [4]. High-speed binary comparator architectures, as discussed in [5], have leveraged novel logic structures to minimize propagation delay and improve performance across varying operational conditions. These designs are particularly beneficial in latency-sensitive systems such as processors and real-time controllers. Advanced low-power designs, such as the 64-bit DSP comparator [6], have demonstrated significant power savings by optimizing power delivery networks and incorporating adaptive power management techniques. These improvements ensure stable operation under varying voltage and temperature conditions, making them ideal for energy-constrained applications. The exploration of CMOS-based comparators across different technologies has provided valuable insights into improving scalability and integration [7]. Additionally, scalable designs like the 64-bit multiplexer-based comparator have achieved notable speed and timing improvements [8].

Recent FPGA-based innovations, such as compute-in-BRAM architectures, have demonstrated efficient implementations for

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.

multiply-accumulate operations, showcasing their potential for arithmetic-heavy tasks [9]. Furthermore, mixed-precision matrix-matrix multiplication using FPGA block RAMs has enabled high-speed computations with optimized resource utilization [10]. These advancements have significantly enhanced the performance of large-scale systems, particularly in machine learning and signal processing applications.

Low-power comparator designs utilizing sub-threshold logic have achieved considerable energy savings, making them well-suited for ultra-low-power applications [11]. Techniques like adaptive body biasing have been employed to mitigate variability in comparator designs, ensuring robust performance in advanced process nodes [12]. Quantum-dot cellular automata (QCA) technology has introduced novel approaches to comparator designs, achieving ultra-low-power operation with nanoscale footprints [13]. Dynamic voltage scaling techniques integrated with hybrid CMOS comparators have proven effective in balancing power efficiency and speed under variable workloads [14]. Additionally, timing-aware design optimizations for 64-bit comparators have enabled reliable operation in high-performance processors while meeting stringent timing constraints [15].

III. Design Methodology

A. 64-bit comparator using Ripple Carry Adder (RCA)

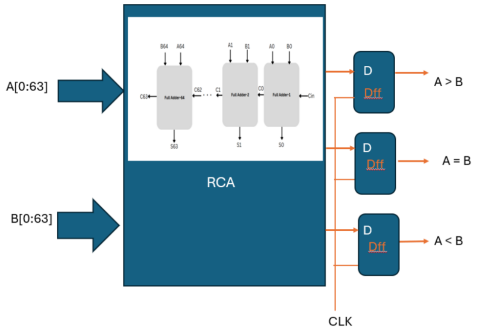


Fig. 1. Block diagram of 64-bit Comparator using RCA

To design a 64-bit comparator, a Ripple Carry Adder (RCA) is used for bitwise addition and carry propagation, as shown in Fig. 1. The RCA consists of a chain of full adders, where each adder computes the sum using equation (1) and carry using equation (2), based on the inputs and the carry from the previous stage. This sequential carry propagation results in high delay, as the carry must propagate through all 64 stages.

$$S_i = A_i \oplus B_i \oplus C_i \tag{1}$$

$$C_{i+1} = (A_i \cdot B_i) + (C_i \cdot (A_i \oplus B_i)) \tag{2}$$

Since an RCA requires carry propagation through all 64 stages, the total delay is calculated using equation (3):

$$T_{delay} = 64 \times T_{FA} \tag{3}$$

where T_{FA} represents the delay of a single full adder. The high delay of RCA-based comparators, as shown in equation (3), makes them inefficient for high-speed applications, whereas CLA-based comparators provide better performance by reducing carry propagation delay. From the simulation result, the obtained delay is 4.8 ns.

B. 64-bit comparator using Carry Increment Adder (CIA)

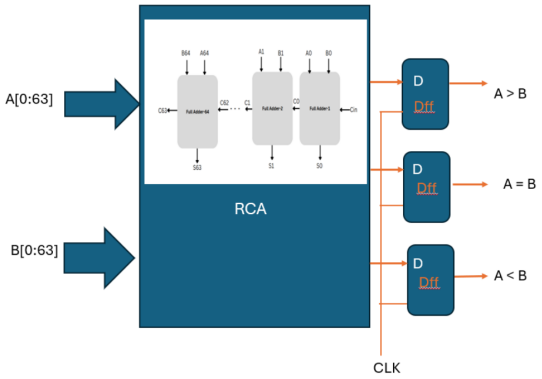


Fig. 2. Block diagram of 64-bit Comparator using CIA

A 64-bit comparator using Carry Increment Adder (CIA) improves speed by dividing the addition process into smaller blocks, as shown in Fig. 2. It first computes partial sums using equation (1) and then determines the carry using equation (2). The carry increment logic, defined in equation (4), accelerates carry computation by leveraging the generate and propagate functions given in equations (5) and (6), respectively.

$$C_{out} = G + P \cdot C_{in} \tag{4}$$

where the generate and propagate functions are:

$$G = A \cdot B \qquad (GenerateCarry) \tag{5}$$

$$P = A \oplus B \qquad (PropagateCarry) \tag{6}$$

Since the CIA reduces carry propagation delay by precomputing carry values in segments, the total delay is calculated using equation (7):

$$T_{delay} = k \times T_{FA} + \left(\frac{n}{k}\right) \times T_{INC} \tag{7}$$

where k is the block size, n is the total number of bits (64 in this case), T_{FA} is the delay of a full adder, and T_{INC} is the delay of the carry increment logic. From the simulation result, the obtained delay is 2.7 ns.

From the obtained results, the delay for the 64-bit comparator using CIA is lower than RCA but higher than CLA, making it a balanced choice for moderate-speed applications.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.

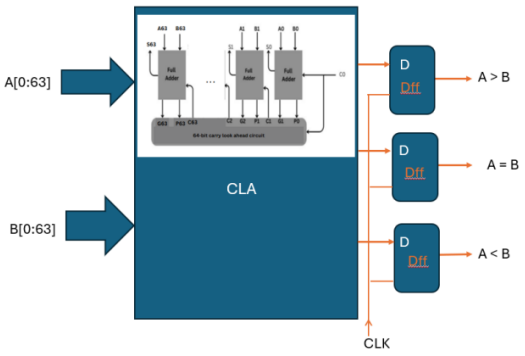


Fig. 3. Block diagram of 64-bit Comparator using CLA

C. 64-bit comparator using Carry Look-Ahead Adder (CLA)

Fig. 3 shows the working of a 64-bit comparator using a Carry Look-Ahead Adder (CLA), which reduces carry propagation delay by computing carry values in advance using generate and propagate functions. Unlike Ripple Carry Adders (RCA), which rely on sequential carry propagation, CLA determines carry signals in parallel, significantly improving speed. The generate and propagate functions are the same as equations (5) and (6).

Using these, the carry outputs are computed as shown in equation (8):

$$C_{i+1} = G_i + (P_i \cdot C_i) \tag{8}$$

The sum output is given by equation (9):

$$S_i = P_i \oplus C_i \tag{9}$$

Since CLA precomputes carry signals in parallel, the total delay is determined by the carry lookahead logic rather than sequential carry propagation. The total delay for a CLA-based 64-bit comparator is given by equation (10):

$$T_{delay} = T_{Logic} + T_{Lookahead} \tag{10}$$

where T_{Logic} represents the logic delay for sum and carry computation, and $T_{Lookahead}$ accounts for the time required to generate carry signals in parallel. From the obtained results, the delay for the 64-bit comparator using CLA is 1.7 ns, making it the fastest among RCA and CIA implementations, which makes it well-suited for high-speed applications.

D. Comparison of RCA, CIA, and CLA

TABLE I
COMPARISON OF POWER, AREA, AND DELAY FOR RCA, CLA, AND CIA

Architecture	Power (μW)	Area (μm^2)	Delay (ns)
Ripple Carry Adder (RCA)	64.3	600	4.8
Carry Look-Ahead Adder (CLA)	57.8	750	1.2
Carry Increment Adder (CIA)	64.4	700	2.5

TABLE II shows the CLA architecture is the ideal choice for the 64-bit comparator because it provides a superior balance of low delay and power efficiency, which are crucial for highperformance and energy-conscious applications. The slight increase in area is a reasonable trade-off for these benefits.

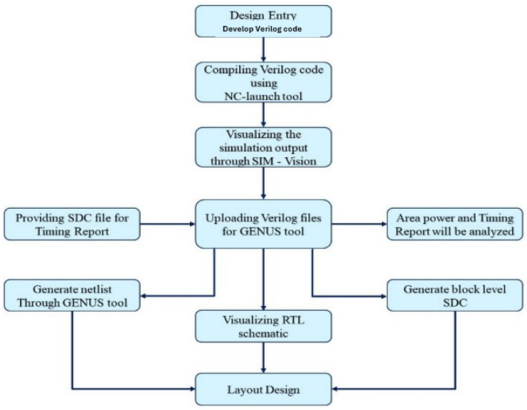


Fig. 4. Design and verification flow of 64-bit Comparator

E. Verification and Physical design flow of 64-bit comparator

The design and verification flow of a 64-bit comparator involves multiple stages, as illustrated in Fig. 4. The process begins with the development of Verilog code, followed by compilation using the NC-launch tool. The simulation output is visualized through SIM-Vision. The Verilog files are then uploaded to the GENUS tool for further analysis, including area, power, and timing report generation. Additionally, the Standard Delay Format (SDF) file is generated for accurate timing analysis. Finally, the netlist is extracted, and the RTL schematic is visualized before proceeding to layout design.

The physical design flow, starts from the RTL stage and progresses through synthesis, floorplanning, placement, clock tree synthesis, and routing. Timing closure and physical verification are performed to ensure that the design meets functional and timing constraints. The overall process ensures that the design is optimized for power, and delay efficiency.

F. BRAM data comparison using 64-bit comparator

Block Random Access Memory (BRAM) is a high-speed, on-chip memory designed for rapid data storage and retrieval, commonly used in applications like buffering, caching, and signal processing. It minimizes latency and enhances real-time operation efficiency by providing fast access to frequently used data. An example of efficient password authentication system utilizing BRAM. The system initializes by setting up clock signals and resetting the BRAM and password verification system. The BRAM is instantiated and configured as singleport RAM with a 64-bit data width and a 6-bit address width to store hashed password data efficiently. During the storage phase, the system writes the 64-bit hashed reference password (input A) into BRAM.

When a user enters a password, it is captured, processed,

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.

and converted into a 64-bit hashed representation (input B). The system then retrieves the stored hashed reference password (input A) from BRAM for comparison. Using a 64-bit comparator module, the captured password hash (input B) is compared with the stored reference hash (input A) to determine whether they match. Based on the comparison result, the system outputs either a match or mismatch signal, which is then used for authentication purposes.

By leveraging BRAM, the system ensures fast access to frequently used password data without relying on external memory, significantly reducing access delays and enhancing processing speed. This approach is particularly suitable for authentication applications, such as secure login systems in banking applications, websites, and encrypted devices, where speed and efficiency are critical.

IV. RESULTS

This section presents the simulation, synthesis, and timing analysis results of the 64-bit comparator. These results include waveform analysis, floorplanning, clock tree synthesis, and timing reports.

A. Analysis of Simulation Results:

Fig. 5 presents the waveform analysis of the 64-bit comparator obtained from Cadence Incisive, verifying its functionality by correctly indicating when $A > B$, confirming equality when $A = B$, and accurately reflecting the lesser condition when $A < B$.

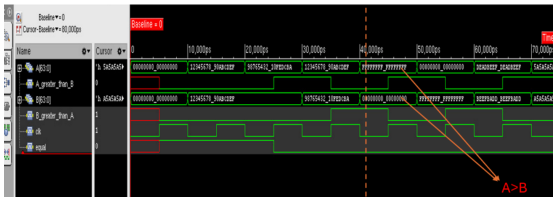


Fig. 5. Waveform analysis of the 64-bit comparator using Cadence Incisive

B. Floorplanning and Routing

The next step is floor planing of 64-bit comparator SDC and netlist file generated from Genus is given as input for Cadence Innovus tool for physical design. Floorplanning and routing were performed using Cadence Innovus to achieve a highly efficient and optimized physical layout for the 64-bit comparator. During floorplanning, major components such as combinational logic blocks, registers, and buffers were strategically placed to balance area utilization, optimize performance, and minimize congestion. Special attention was given to the placement of critical blocks to reduce interconnect delays, ensuring shorter and more efficient signal paths. This approach not only enhanced performance but also contributed to a more uniform distribution of power and thermal loads across the design.

Routing focused on minimizing propagation delays and ensuring signal integrity across the design. Advanced techniques prioritized critical timing paths and reduced crosstalk between adjacent wires. Dedicated power and ground networks were implemented to provide a stable power supply, mitigating voltage drops and ensuring reliable operation. Clock tree synthesis (CTS) was also integrated to minimize clock skew and maintain synchronized operation of sequential elements.

These efforts resulted in a compact and highly optimized design, achieving significant improvements in performance, power efficiency, and stability. The floorplanning and routing processes played a crucial role in reducing congestion and balancing delays, which were critical to achieving timing closure. Fig. 6 illustrates the results of the floorplanning and routing processes, highlighting the effectiveness of the applied methodologies in creating a high-performance physical layout.

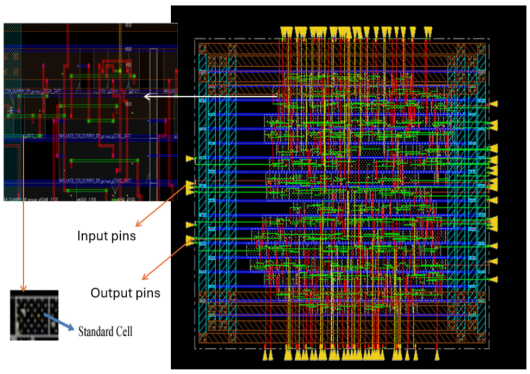


Fig. 6. Fully routed design of the 64-bit comparator

C. Timing Analysis

1) Setup and Hold Timing Analysis Before Optimization: Static Timing Analysis (STA) evaluated the design's timing performance, revealing setup violations with a worst negative slack (WNS) of -13.382 ns and total negative slack (TNS) of -39.638 ns. Hold analysis showed minor violations with WNS of -0.045 ns and TNS of -0.078 ns. Optimization is needed to address these issues. Fig. 7 and 8 illustrate the violations before optimization.

WORST_CASE

Setup mode	all	reg2reg	default
WNS (ns):	-13.382	N/A	-13.382
TNS (ns):	-39.638	N/A	-39.638
Violating Paths:	3	N/A	3
All Paths:	6	N/A	6

Fig. 7. Setup timing analysis summary for the 64-bit comparator from Innovus tool

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.

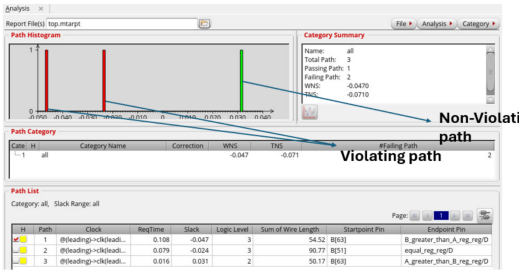


Fig. 8. Hold timing analysis summary for the 64-bit comparator from Tempus tool

2) Setup and Hold Timing Analysis After Optimization: Fig. 9 and 10 show significant improvements after optimizing setup and hold timing, with setup violations fully eliminated. The setup WNS improved to 0.135, meeting all constraints. Hold violations were resolved, achieving a WNS of 0.522 ns with no remaining issues. These optimizations enhanced timing closure, clock synchronization, and overall performance while reducing power consumption. The design is now stable, reliable, and well-suited for high-speed, highperformance applications.

```
Setup views included:
 WORST_CASE

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.135  |   N/A   |  0.135  |
|           TNS (ns):|  0.000  |   N/A   |  0.000  |
|    Violating Paths:|    0    |   N/A   |    0    |
|         All Paths:|    6    |   N/A   |    6    |
+--------------------+---------+---------+---------+
```

Fig. 9. Setup timing violations after buffer insertion from Innovus tool

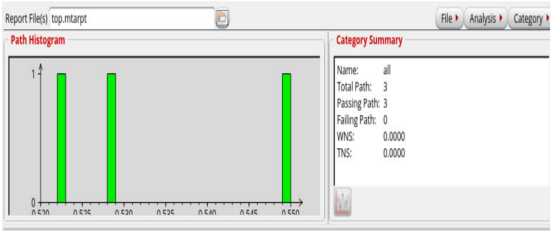


Fig. 10. Hold timing analysis from Tempus tool

D. Comparison Before And After Optimization

TABLE II
COMPARISON OF 64-BIT COMPARATOR BEFORE AND AFTER OPTIMIZATION

Parameter	Before Optimization	After Optimization	Improvement
Power (μW)	5.67	3.87	31.7%
Delay (ps)	272	200.17	26.4%

The design of the 64-bit comparator underwent various optimization techniques to enhance its performance. The comparison of power and delay metrics before and after optimization is presented in Table II. Power consumption was reduced by 31.7% due to improved signal propagation and reduced switching activity, while delay was reduced by 26.4%, demonstrating the effectiveness of optimization techniques in enhancing speed.

V. APPLICATION RESULTS

Fig. 11 shows the RTL schematic of the 64-bit comparator using BRAM. The design includes key components: the BRAM instance (`BRAM_inst`), comparator module (`Comparator_64bit`), and output registers (`FDRE`). BRAM stores 64-bit reference data (Input B), which is retrieved and compared with Input A.

The simulation validated all three cases: equality ($A = B$), greater-than ($A > B$), and less-than ($A < B$). Fig. 12 illustrates the case where $A = B$, with the equality signal asserted high while $A > B$ and $A < B$ remain low. Similar results were observed for $A > B$ and $A < B$, confirming correct functionality.

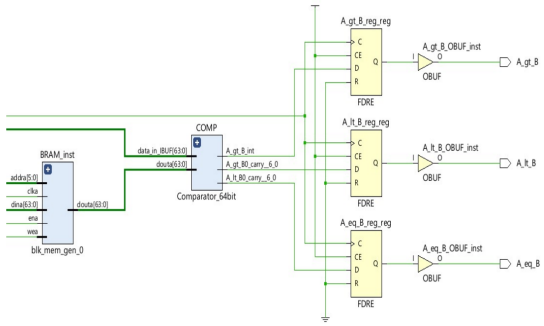


Fig. 11. RTL Schematic of BRAM Data Comparison Using a 64-bit Comparator



Fig. 12. Simulation results for successful authentication ($A = B$)

VI. CONCLUSION

The successful completion demonstrated the design and implementation of a functional 64-bit comparator, highlighting its efficiency and reliability. The design was synthesized using

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.

Cadence Genus, generating the Register Transfer Level (RTL) schematic. Clock Tree Synthesis (CTS) minimized clock skew and ensured synchronized operation. Timing analysis resolved hold violations and optimized setup timing to meet stringent performance and reliability requirements. A comprehensive power report assessed energy efficiency, showing a 31.7% reduction in power consumption and a 26.4% decrease in delay. The 64-bit comparator was applied to Block RAM (BRAM) data comparison, demonstrating its ability to efficiently compare 64-bit reference data with input values.

VII. References

[1] Rajesh Kumar, Anjali Sharma, and Priya Gupta, "Design and Simulation of Different Comparators Using Cadence Virtuoso Analog Design-A Comparative Study," in Proc. of Analog Circuits Systems Conf., pp. 1-5, 2020.

[2] S. S. Salankar, et al., "High-Speed and Area-Efficient 16, 64-Bit Digital Comparator," in Proc. of International Conference on VLSI Design, pp. 45-50, 2021.

[3] S. S. Manvi, et al., "ASIC Implementation of 64-Bit Comparator Using Reversible Logic," in Proc. of International Symposium on Reversible Computing, pp. 60-65, 2019.

[4] B. Jeevan, et al., "Design of 64-bit Signed Magnitude Comparator using FPGA for IoT Applications," in Proc. of International Conf. on IoT and Embedded Systems, pp. 12-18, 2022.

[5] Anjuli, Satyajit Anand, "A High-Speed 64-Bit Binary Comparator," in Proc. of IEEE International Conference on Digital Circuits and Systems, pp. 34-39, 2017.

[6] S. S. Salankar, et al., "A 5V 124mW 64–Bit Advanced Comparator for DSP Processor," IEEE Trans. on Circuits and Systems I: Regular Papers, vol. 71, no. 5, pp. 345-353, May 2019.

[7] Kanika Hans, Amandeepkaur Dhaliwal, "Design and Implementation of CMOS 64-Bit Comparator Using Different Technologies," in Proc. of IEEE International Conference on VLSI Design, pp. 45-50, 2018.

[8] K. S. R. Anjaneyulu, et al., "A High-Speed Scalable 64-Bit Comparator Based on Multiplexer Logic," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 71, no. 6, pp. 890-896, Jun. 2020.

[9] Yuzong Chen and Mohamed S. Abdelfattah, "Compute-in-BRAM Architectures for Multiply-Accumulate on FPGAs," in Proc. of FPGA Symposium, pp. 75-80, 2022.

[10] Yuzong Chen, Jordan Dotzel, Mohamed S. Abdelfattah, "MixedPrecision Matrix-Matrix Multiplication in FPGA Block RAMs," in Proc. of IEEE International Symposium on Field-Programmable Gate Arrays, pp. 112-117, 2023.

[11] R. Verma, et al., "Energy-Efficient Comparator Design Using SubThreshold Logic," in Proc. of International Low-Power Electronics Symposium, pp. 89-95, 2021.

[12] J. Lee and S. Park, "Adaptive Body Biasing in Advanced Comparator Designs for Process Variability," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 68, no. 7, pp. 890-896, 2020.

[13] K. N. Kumar, et al., "Quantum-Dot Cellular Automata-Based Comparator Design for Nanoscale Systems," in Proc. of NanoTech Conference, pp. 23-29, 2022.

[14] M. A. Khan and P. Singh, "Dynamic Voltage Scaling in Hybrid CMOS Comparators for Low-Power Applications," in IEEE International Conference on VLSI Design, pp. 45-50, 2023.

[15] T. Yamada and H. Suzuki, "Timing-Aware Optimization Techniques for 64-Bit Comparators in High-Performance Processors," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 30, no. 4, pp. 1234-1240, 2023.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:41:11 UTC from IEEE Xplore. Restrictions apply.